scraps-perpair: project overview

Raymundo Arroyave

2026-05-20

Contents

scraps-perpair: project overview **1**
1. What SCRAPS is . . . 1
2. What scraps-perpair adds and fixes . . . 2
3. "Is SCRAPS lucky, or is it actually optimizing?" . . . 6
4. The Python tool suite (`tools/`) . . . 7
5. Test suite . . . 7
6. Validation against ATAT mcsqs . . . 8
7. Release history . . . 8
8. What's next . . . 8

scraps-perpair: project overview

This document describes (a) what SCRAPS is, (b) what was extended in this fork, and (c) how to use the resulting tool. It is meant as a technical map of the codebase for a new collaborator or for our future selves. For installation and quick-start usage see the top-level `README.md`; for benchmarks against ATAT mcsqs see `docs/mcsqs_comparison.md`.

---

1. What SCRAPS is

SCRAPS — **S**uper**C**ell **R**andom **AP**proximate**s** — is a combinatorial-optimization code for generating special quasi-random structures (SQS) and short-range-ordered (SRO) supercells of multicomponent alloys. The upstream version was developed at Iowa State by Rahul Singh, Prashant Singh, Aayush Sharma, Ganesh Balasubramanian, and Duane D. Johnson, and is published in:

R. Singh et al., *Nature Computational Science* **1**, 54 (2021). DOI: 10.1038/s43588-020-00006-7

The upstream repository is `DuaneDJohnson/Hybrid-Cuckoo-Search`. The MIT license has been preserved with dual copyright (Iowa State + Arroyave fork).

1.1 The Warren–Cowley formulation

For a multicomponent alloy on a single sublattice, the Warren–Cowley short-range-order parameter is

$$\alpha_{ij}^{(k)} = 1 - \frac{P(j \mid i,k)}{c_j}$$

where $P(j \mid i,k)$ is the conditional probability that the neighbor of an atom of species i at the k-th coordination shell is of species j, and c_j is the bulk concentration of species j. The "random" limit (an exact SQS) has $\alpha_{ij}^{(k)} = 0$ for all (i,j,k). Ordered structures (B2, $L1_2$, etc.) have characteristic non-zero α signatures at specific shells.

SCRAPS' loss function on a candidate supercell is

$$E = \sum_{k=1}^{K_{\max}} w_k \sum_{i \neq j} \left(N_{ij}^{(k)} - T_{ij}^{(k)}\right)^2$$

where $N_{ij}^{(k)}$ is the actual count of (i,j) pairs at shell k in the candidate and

$$T_{ij}^{(k)} = N_i \cdot (1 - \alpha_{ij}^{(k),\,\text{target}}) \cdot z_k \cdot N_j/N$$

is the target. The fitness reported in the log is $\sqrt{E}$.

1.2 The Hybrid Cuckoo Search

SCRAPS minimizes E over the discrete space of atomic arrangements using a metaheuristic combining:

- **A population of "nests"** (candidate supercells), each independently evolving through MC swap moves (`empty_nests`).
- **A "cuckoo" step** that probabilistically replaces low-fitness nests with new randomized ones (`get_cuckoos`), giving the search a way to escape local minima.
- **Global ranking** across MPI ranks at the end (`get_best_nest` plus the merge step in `main`).

Per-rank MC swap moves are: pick two atom positions, swap their species, evaluate fitness, accept if it improved (with a small ε-band stochastic acceptance to allow neutral moves).

1.3 Original assumptions and limitations

- Single sublattice with uniform per-shell coordination.
- A **single scalar α target** applied to every species pair at every shell (no per-pair targeting).
- POSCAR + INPUT file format (a SCRAPS-specific dialect of VASP POSCAR with no species labels, plus a custom INPUT keyword block).
- MPI-parallel over nest population; no shared memory inside a rank.

---

2. What scraps-perpair adds and fixes

Six bug fixes from the upstream code, a feature extension (per-pair SRO targets), spectator-sublattice support for HEB-class structures, user-facing heuristics, an algorithmic upgrade (delta-fitness MC), a reproducibility layer, and a Python-side tool suite.

2.1 Bug fixes shipped from upstream

#	Location	Description
1	`readPOSCAR`	Uninitialized `totalatoms` (upstream issue #3).
2	`fitnessvalue`	The pair-gating test `nest[center] != nest[surr]` compared species indices stored at *atom* indices, dropping unrelated entries. Replaced with `center != surr`.
3	`printtoscreen`	Wrong Warren–Cowley formula (off by $\sim N_i$, asymmetric in $i \leftrightarrow j$); the printed α didn't match the residual the optimizer was minimizing.
4	`main`	`WEIGHTS` size mismatch claimed to STOP but read past the array end. Now `MPI_Abort` with a clear message.
5	`randomizeLattice`	Infinite loop when `sum(ELEMENT)` $\neq$ `Cell_DIM` $\times$ `atoms_per_basis`. Now guarded with a diagnostic and abort.
6	`Lattice::neighbourMatrix`	Per-atom shell numbering used a `removeduplicates` call inside the outer loop, so floating-point jitter near a shell boundary could assign the same physical neighbor to different shell indices for different atoms. Rewritten with **global shell numbering** at a 2% relative-gap tolerance.

Item #2 (the species-index gating bug) was the silent showstopper — it caused the upstream fitness function to misclassify ~40 % of pairs depending on memory layout, while still appearing to "work" because the resulting structures looked nominally random.

2.2 The headline feature: per-pair SRO targets

New INPUT keyword:

```
SRO  <species_i>  <species_j>  <shell_index>  <alpha_target>
```

Sets $\alpha_{ij}^{(k)\,\mathrm{target}}$ for one (i, j, k) triple. Lines may repeat; entries are automatically symmetrized $\alpha_{ji} = \alpha_{ij}$. Pairs not mentioned default to $\alpha = 0$ (the SQS target).

The targetSROpair tensor is threaded through `fitnessvalue`, `fitnessofnests`, `get_best_nest`, `empty_nests`, `empty_nests_fast`, `get_cuckoos`, and `main`. Upstream only supported a single scalar α applied uniformly; this fork lets each species pair at each shell have its own target.

This is what makes targeting **classical ordered structures** in a single SCRAPS run possible. See `tests/{b2_bcc_binary,l12_fcc_a3b, l10_fcc_ab,d03_bcc_a3b,l11_fcc_ab,l21_bcc_x2yz}/` for the six worked cases (B2, $L1_2$, $L1_0$, $D0_3$, $L1_1$, $L2_1$/Heusler).

2.3 Spectator sublattice (HEB-class structures)

For structures like AlB_2-type (Hf,Zr,Ti,Ta,Nb)B_2 high-entropy diborides, one sublattice is compositionally fixed (boron) and the other carries the chemical disorder. New INPUT keyword:

```
SPECTATOR_BASIS_SPECIES  <basis_index>  <species_index>
```

declares that all atoms at the given POSCAR basis position are fixed at the given species and not touched by MC. The fitness function becomes "M-sublattice only" — pairs involving a spectator atom are excluded, the reference concentration becomes N_j^M/N^M, and the effective coordination is the count of non-spectator neighbors per non-spectator atom.

A coordination check (`Lattice::coordination_per_shell`) is run at startup on the non-spectator subset; if it's non-uniform, SCRAPS aborts with a diagnostic pointing to ATAT mcsqs for the truly multi-sublattice cases SCRAPS can't handle.

See `tests/heb_5metal_random/` and `tests/heb_5metal_csro/` for worked HEB cases.

2.4 User-facing heuristics

Keyword	Purpose
`AUTO_MCSTEP <k>`	Override `MCSTEP` with `max(50, k · N)` once supercell size is known, removing the manual tuning. Default factor 7.
`EARLY_STOP_PATIENCE <n>`	Stop the outer optimization loop once the per-rank best fitness has not improved by more than `EARLY_STOP_TOL` for `n` consecutive iterations. Default 0 = disabled.
`EARLY_STOP_TOL <tol>`	Improvement threshold for the patience counter. Default 1e-6.
`FAST_SWAPS <0|1>`	Delta-fitness MC path (see 2.5). Default 1.
`TARGETED_SWAPS <0|1>`	Residual-biased swap proposals (see 2.5). Default 0 (opt-in).
`TARGETED_SWAP_PROB <p>`	Bias level for targeted swaps. Default 0.7.
`RNG_SEED <n>`	Reproducible per-rank seeding via $n \oplus (\text{rank} \cdot \phi_{64})$. Default 0 = use `std::random_device`.
`RANDOM_ONLY <0|1>`	Skip the MC loop and return the best of the initial random nest pool. Default 0.
`CONVERGENCE_LOG <path>`	Rank-0 per-iteration CSV log: `iteration, best_fitness, wall_seconds`. Iteration −1 is the pre-MC baseline.

Keyword	Purpose
`INIT_NEST <path>`	Warm-start: read a plain-text file of species indices (one per line, in basis-OUTERMOST atom order) and seed `nests[0]` on every rank with that nest. The MC then explores around the seed instead of from a uniform random init. Useful for steep multi-pair targets ($L2_1$ etc.) and iterative-refinement workflows. Default empty = pure random init.

The first three let users get to a sensible result without per-cell tuning. The reproducibility / ablation knobs (RNG_SEED, RANDOM_ONLY, CONVERGENCE_LOG) exist primarily to make the optimizer *auditable* — see §3. `INIT_NEST` lets users explicitly hand the optimizer a candidate structure as a starting point.

2.5 Algorithmic upgrades

Delta-fitness MC (`empty_nests_fast`, default on). The legacy `empty_nests` recomputed the full $O(N \cdot z)$ fitness on every swap proposal. The fast path precomputes the constant target matrix $T_{ij}^{(k)}$ once per nest, maintains the pair-count tensor $N_{ij}^{(k)}$ as state, and applies an $O(z)$ delta per swap (touching only the neighbors of the two swapped atoms; the swapped pair itself contributes zero net delta by symmetry). On RNG-pinned A/B tests against the legacy path, this yields **2–18× speedup on loop-bound cases** with identical final fitness:

Case	Legacy wall	Fast wall	Speedup
25:50:25 n=108 (plateau)	15.2 s	0.8 s	18.6×
25:50:25 n=256	3.1 s	0.4 s	8.2×
BCC 50:50 n=54	4.8 s	0.6 s	7.4×

Targeted swap proposals (`empty_nests_fast`, off by default). With probability `TARGETED_SWAP_PROB`, propose swaps biased toward the species pair (i, j, k) with the largest weighted residual[2]. The remaining fraction stays uniform-random for exploration. **Honest finding from our benchmarks: on the test cases we measured (cell sizes up to n=500, B2 / $L1_2$ / $L1_0$ / $D0_3$ / $L1_1$ ordered targets, infeasibility plateaus), uniform-random already converges fast enough that the bias adds overhead without speedup.** Kept as an opt-in for workloads where uniform random would genuinely struggle (very large supercells, complex multi-pair SRO, highly non-convex landscapes) and as a starting point for further algorithmic experimentation.

2.6 Feasibility check (the "is fitness=0 even achievable?" math)

For an SQS target ($\alpha = 0$), the expected pair count $T_{ij}^{(k)} = N_i \cdot z_k \cdot N_j/N$ must be **integer** for every weighted shell-pair. If it isn't, the fitness has a strictly positive lower bound — the optimizer plateaus at a non-zero value no matter how long it runs.

This caught us by surprise during the cell-size benchmark: 25:50:25 on FCC k×k×k is feasible for k even and infeasible for k odd (at shell 1, $T_{AC} = 3k^2/2$), explaining the deterministic `fitness = √2.5 ≈ 1.58` plateau at n=108 and n=500.

`tools/suggest_cell.py` walks candidate cell sizes (including non-cubic supercells) and returns the closest feasible `Cell_DIM`, plus the fitness floor for infeasible candidates. It handles HEB-style spectator sublattices via the same M-only math SCRAPS uses internally. The C++ runtime also runs the feasibility check at startup and prints a warning naming the offending pair(s) when the configured `(ELEMENT, Cell_DIM, MAXSHELLNUM)` is infeasible.

2.7 MPI singleton-mode diagnosis

The macOS-with-Homebrew-Open-MPI setup commonly fails to form a real multi-rank communicator when `mpirun` from a different MPI install (e.g. anaconda's MPICH Hydra `mpirun`) is first on `$PATH`. Each process then runs as a singleton with rank 0 and `nproc=1`, the 4 "ranks" do independent searches, and only one's `SCRAPS.vasp` survives (the last to write).

Three coordinated fixes: 1. **C++ runtime detects the condition** (`nproc==1` but `$OMPI_COMM_WORLD_SIZE > 1`) and prints a loud one-shot warning with the fix. 2. **`tools/tests` and `tools/run_sqs.py` auto-resolve** the matching `mpirun` by looking next to the `mpic++` from `build.sh` (`$MPICXX`) before falling back to PATH. 3. **README section** documenting the gotcha.

When real multi-rank can't be achieved, the conv-log path is suffixed with the OS PID so concurrent singletons don't race-and- interleave their writes (the test helper handles both modes).

---

3. "Is SCRAPS lucky, or is it actually optimizing?"

The reproducibility / ablation keywords in §2.4 exist mainly to answer this question rigorously. The experiment:

Case	metric	Random-only (best of 96)	SCRAPS (full MC)	improvement
25:50:25 n=108 SQS plateau	fitness	~20 ± 3	1.58 (all 3 seeds)	**~12×**
25:50:25 n=108	max|α|	0.14–0.26	0.012	**~15×**
B2 n=128 ordered	fitness	369–403	0 (perfect B2)	∞
B2 n=128	search-space prob	$\sim 10^{-35}$ per nest	finds in ~1 s	qualitative

The B2 case is the clincher: random search has $\sim 10^{-37}$ chance per random nest of containing a B2 ordering. SCRAPS finds it deterministically. The MC search is doing real combinatorial optimization, not just shuffling.

Reproducible via `docs/baseline_vs_scraps.sh`.

---

4. The Python tool suite (`tools/`)

Script / module	Purpose
`verify_sro.py`	Independent pymatgen-based α-matrix verifier. Supports multi-shell, spectator-aware HEB mode, target-file pass/fail comparison, JSON output for CI. Used as the cross-check on every `SCRAPS.vasp` we generate.
`suggest_cell.py`	Picks the smallest feasible Cell_DIM near a user-supplied target atom count for a given composition. Supports cubic-only or non-cubic search, HEB spectator sublattices, and diagnose mode for a single `(ELEMENT, Cell_DIM)` configuration.
`run_sqs.py`	One-shot wrapper: composition + target atoms → suggest_cell pick → INPUT with sensible heuristic defaults → mpirun → result. Handles HEB targets via `--spectator-basis-species`.
`scraps_ase.py`	ASE interoperability: `run_sqs_from_atoms` takes an ASE `Atoms` primitive in and returns an SQS `Atoms` out; `read_scraps_output` / `write_scraps_poscar` / `make_scraps_inputs` are lower-level helpers. Handles the SCRAPS vs canonical-VASP POSCAR layout difference internally.

All four are pymatgen-and/or-ASE-based and run completely outside the C++ code. `verify_sro.py` is in particular the canonical "did SCRAPS actually achieve what it claims?" cross-check.

---

5. Test suite

`tools/tests/`, run with `pytest`. **48 tests, ~34 s locally**, all with `RNG_SEED` pinned so the optimizer behavior is reproducible:

- `test_verify_sro.py` (19 tests): verifier against analytically-known structures (B2 NiAl, $L1_2$ Ni_3Al, random binary, HEB random and CSRO).
- `test_suggest_cell.py` (16 tests): feasibility math across cubic and non-cubic FCC/BCC supercells, HEB spectator-aware cases.
- `test_run_sqs.py` (4 tests): end-to-end smoke + INPUT generation.
- `test_scraps_optimization.py` (10 tests): full mpirun-based optimization regressions — feasible SQS hits 0, infeasible plateau hits the predicted floor, B2 SRO target reaches max ordering, RANDOM_ONLY worse than SCRAPS, FAST_SWAPS matches legacy fitness, convergence trajectory is monotone non-increasing, spectator HEB completes correctly.
- `test_scraps_ase.py` (8 tests): ASE I/O round-tripping and the end-to-end ASE wrapper on FCC SQS and BCC B2.

End-to-end tests skip gracefully when the scraps binary or mpirun isn't available, so a fresh checkout passes `pytest` even before `./build.sh`. CI (`.github/workflows/test.yml`) installs `openmpi-dev` and `libeigen3-dev`, builds scraps, and runs the full suite on Ubuntu against Python 3.11 and 3.12 — so the regression net catches optimizer changes, not just verifier-math changes.

---

6. Validation against ATAT mcsqs

`docs/mcsqs_comparison.md` documents head-to-head comparisons against ATAT 3.50's mcsqs across three system classes:

- 108-atom NiCoCr FCC SQS (multi-shell), single-sublattice stoichiometry sweep (50:50 / 75:25 / ternary on FCC and BCC),
- FCC cell-size sweep (32 / 108 / 256 / 500 atoms) at 25:50:25 ternary,
- HEB (Hf,Zr,Ti,Ta,Nb)B_2 at 150 and 600 atoms.

Summary verdict: SCRAPS matches or beats mcsqs in α-matrix quality on every case at comparable wall budgets, and is consistently faster (5–60× wall time, 1.5–15× CPU time) because of the explicit α-residual loss versus mcsqs's correlation-engineering approach.

The per-pair SRO targeting (§2.2) is the unique value-add — mcsqs has no direct API for per-species-pair α targets.

---

7. Release history

- **v0.3.0** (this commit) — warm-start API (`INIT_NEST`), JOSS paper draft under `paper/`, delta-vs-legacy fitness disagreement fix (the $L2_1$ warm-start case where the perfect Heusler nest has zero X-Y and X-Z 2NN pairs, exposing a `countgr==0` edge case in the legacy fitness). 80 tests passing.
- **v0.2.2** — INPUT parser hardening: unrecognized-keyword warnings, `#` comment/blank-line support, required-keyword validation, ELEMENT-vs-TYP cross-check, `Cell_DIM=3` validation; dead `removedublicates` removed. 73 tests.
- **v0.2.1** — Real VASP POSCAR parser (handles full canonical VASP-5 layout, defensive abort on malformed input); ASE coordinate bugfix in `tools/scraps_ase.py`; three Jupyter tutorials (`notebooks/01..03`); `docs/OVERVIEW.{md,tex,pdf}`. 60 tests.
- **v0.2.0** — Per-pair SRO API, HEB-class spectator sublattice support, user-facing heuristics (AUTO_MCSTEP, EARLY_STOP_*, RNG_SEED, RANDOM_ONLY, CONVERGENCE_LOG, FAST_SWAPS, TARGETED_SWAPS), delta-fitness MC (2-18× speedup), feasibility check (`tools/suggest_cell.py` + C++ warning), ASE interop (`tools/scraps_ase.py`), MPI singleton-mode diagnosis and fix. 48 tests.

8. What's next

Open items as of v0.3.0: - **Apply to a real research case.** Refractory HEB with literature CSRO targets, DFT validation. The infrastructure is in place; the work to do it is not. - **Make the repo public + submit JOSS paper.** The manuscript draft under `paper/` is submission-ready; gating items are flipping the repo to public visibility and ORCID verification. - **Sigma-phase / Heusler with independent compositions.** The only remaining functional gap vs ATAT `mcsqs` is true multi-sublattice with independent designed compositions on each sublattice. - **Upstream a subset of fixes.** The six C++ bug fixes (§2.1) and

the POSCAR/INPUT parser hardening are upstreamable as PRs to `DuaneDJohnson/Hybrid-Cuckoo-Search`. - **`tools/plot_convergence.py`** — small matplotlib helper that reads `CONVERGENCE_LOG` and plots fitness vs iteration. Shipped with v0.3.0.

See the `## 13. scraps-perpair` entry in the project memory catalog for the live status of these items.